EMBER NETWORK · TECHNICAL SPECIFICATION

EmberDHT

Protocol Specification

WIRE VERSION	SOFTWARE	TRANSPORT	DATE
2 (min 2)	Ember 1.5.4	Noise_IK / Noise_XX	August 2026

> EmberDHT is Ember's native Kademlia overlay: a signed, encrypted, server-less distributed hash table that Ember nodes run among themselves, in parallel with the eMule KAD network and eD2K servers. This document specifies the identity model, Noise transport, wire frames, routing table, record formats, lookup and publish algorithms, bootstrap paths, and abuse limits as implemented in Ember 1.5.4. It is written from the code, not from comments.

Canonical implementation: `src-tauri/src/network/ember/`

Contents

This specification describes the protocol as shipped in Ember 1.5.4 (`EMBER_DHT_VERSION = 2`). Remaining work, including the dormant native transfer path, is called out where it affects interoperability. Design notes that are not yet wire-stable live in `docs/ember-dht.md`.

SECTION 1

Introduction

What EmberDHT is, what it is not, and the constraints that shaped it.

Ember is a modern eMule-compatible P2P client. Beside speaking KAD and eD2K, every Ember node also runs **EmberDHT**: its own encrypted Kademlia overlay, used to find other Ember nodes, publish shared files, and resolve download sources without a directory server, tracker, or shipped seed list.

1.1 Purpose

EmberDHT answers three questions among Ember-capable peers:

1. **Who is on the overlay?** — routing-table contacts bound to Ed25519 / X25519 keypairs.
2. **Which files exist under a name?** — keyword records, findable by search.
3. **Who can serve this file?** — source records keyed on the eD2K file hash.

Content still moves over the eMule client-to-client wire. EmberDHT discovers the source; eD2K transfers the bytes. A 256 KiB chunk protocol with a BLAKE3 hash tree exists in `ember/transfer.rs` but is not yet wired in.

1.2 Relationship to KAD and eD2K

EmberDHT is a *second network*, not a replacement. It is always on (`ember_native_enabled`). The settings switch remains visible but cannot be turned off. The overlay rides the same UDP socket as KAD (port 4672 by default) and is demultiplexed by a two-byte magic, so one forwarded port serves both stacks.

KAD and eD2K are also how a cold Ember node *finds* the overlay. There is no Ember bootstrap server. Join paths are the KAD rendezvous key, peers noticed in ordinary KAD traffic, Ember-capable eD2K sessions, gossip, and a persisted contact file. The Friends rendezvous server has no role in DHT bootstrap: a server-hosted pool would hand the operator an identity-to-IP roster of every participant.

1.3 Design principles

- **Cryptographic identity.** A node ID is not chosen; it is `BLAKE3(Ed25519 public key)[..16]` . Gossip that claims an ID without the matching key is discarded.
- **Signed everything.** Every DHT frame is Ed25519-signed over its header and payload. Every stored record is signed by its publisher. A `DhtMessage` only exists in verified form.
- **Encrypted transport.** After the magic bytes, the datagram is Noise (ChaCha20-Poly1305, BLAKE2s). Cleartext DHT never leaves the host.
- **No central roster.** Seed lists, DNS SRV, and rendezvous-hosted bootstrap pools are explicitly not planned.
- **Fit the datagram.** Replies are packed to an unfragmented 1400-byte budget. An oversized reply that the peer drops before decryption looks like success to the sender and silence to the receiver.

- **Tighten, never loosen.** Abuse limits start permissive on a near-empty table and converge on KAD-strict values as verified contacts grow. Tightening refuses new admissions; it never evicts what an earlier tier allowed.

1.4 Document conventions

Integers on the DHT frame header and most payload lengths are **little-endian**. Socket ports in contact addresses are **big-endian**. Byte strings are concatenated in the order shown. Hexadecimal constants use a `0x` prefix. Time constants are wall-clock seconds unless noted as `Instant` (process-monotonic).

> **NORMATIVE SOURCE**
>
> Where this document and a comment disagree, the constants and encoders in `src-tauri/src/network/ember/` win. Protocol constants live in `dht/mod.rs` and `dht/messages.rs`; publish/search drivers and maintenance live in `network/mod.rs`.

2. Architecture

Ember frames share the KAD UDP socket. The network task demultiplexes on magic, decrypts through the Noise session table, then offers the plaintext to the control decoder first and the DHT decoder second. The two namespaces cannot alias: control version `0xC1` sits far outside the DHT version range.

APPLICATION	Search UI · Library publish badges · Ember Network page · status-bar gauges
DRIVERS	**Publish / search / maintenance** in `network/mod.rs` — keyword & source cycles, KAD bridge, rendezvous lookup, streaming results
DHT ENGINE	**IO-free protocol core** — routing table, store, iterative search, signed frame build/parse. Transport-agnostic: it consumes decrypted bytes and returns frames to encrypt.
RECORDS	Keyword (`0x01`) and source (`0x02`) blobs, Ed25519-signed by the publisher, stored under 16-byte BLAKE3 keys
TRANSPORT	**Noise_IK** when the peer's static X25519 is known · **Noise_XX** for first contact with a stateless retry cookie · ChaCha20-Poly1305 · BLAKE2s
UDP	Shared KAD socket · magic `0xEB 0x3E` · default port **4672** · max datagram 4096 B, replies packed to 1400 B

Figure 1. EmberDHT stack. The engine is deliberately free of sockets so the protocol is unit-testable without a live network.

2.1 What the overlay does and does not do

DOES	DOES NOT
Find Ember nodes and keep a verified routing table	Move file bytes (that is still eD2K c2c)
Publish and look up keyword and source records	Replace KAD or eD2K; it runs beside them
Carry BLAKE3 integrity digests on records	Host a bootstrap roster or seed list
Let firewalled nodes publish via buddy `PROXY_STORE`	Let a searcher callback a firewalled source (consume side is unfinished)
Refuse incompatible wire versions at the version byte	Negotiate an upgrade or tell the user why a peer was dropped

2.2 Always on

Profiles that still had the overlay off are turned on at load. Publishing starts once the node has contacts. A fresh node can sit at zero contacts for a minute or two while the first maintenance cycle runs the KAD bridge; the Ember Network page reports **Connecting** for that window rather than a fault.

3. Identity and cryptography

3.1 Keypairs

Every Ember node holds two static keypairs, generated once and persisted:

KEY	ALGORITHM	ROLE
`Ed25519`	ed25519-dalek, strict verification	Node identity, frame signatures, record signatures, node ID derivation
`X25519`	static Noise key	Transport session (IK when known, XX when not)

3.2 Node ID

```
node_id[16] = BLAKE3(ed25519_public_key[32])[0..16]
```

The 128-bit ID is compatible with Ember's existing `ember_hash` field and is cryptographically bound to the signing key. A peer-supplied `node_id` in a contact list is never trusted: the receiver re-derives it from the Ed25519 bytes and refuses the contact if the key is not a valid curve point or the ID does not match. This is the primary defence against routing-table poisoning.

> **PROOF OF POSSESSION**
> The BLAKE3 binding is an *offline consistency check*, not proof the peer holds the private key. Anyone who observed a legitimate public key on the wire can replay the pair. Frame and record signatures are the proof-of-possession for DHT work. Friend-level privileges use a separate challenge-response over a fresh nonce and are out of scope here.

3.3 Frame signature

`encode_message` serializes version, type, request id, sender id, optional public key, payload length, and payload, then appends a 64-byte Ed25519 signature over every preceding byte. `decode_message` verifies that signature and the `sender_id == BLAKE3(pubkey)[..16]` binding *before* constructing a `DhtMessage`. An invalid frame never becomes a message object.

3.4 Record signature

A stored record is a publisher-signed blob. The signature covers the record body (type, hashes, size, publisher key, timestamp, name, and for sources the contact block). Storers re-send the identical bytes; they do not re-sign. Expiry is computed from the *signed* creation timestamp, so replication cannot extend a record's life.

4. Transport

Ember UDP payloads are told apart from KAD/eD2K by magic `0xEB 0x3E`. Everything after the magic is encrypted. Patterns:

- `Noise_IK_25519_ChaChaPoly_BLAKE2s` — peer's static X25519 is already known (routing-table contact, previous session).
- `Noise_XX_25519_ChaChaPoly_BLAKE2s` — first contact, including the eD2K-bridge path where no static key is known.

4.1 Packet types

TYPE	NAME	ROLE
0x01	PKT_IK_INIT	IK handshake, initiator → responder
0x02	PKT_IK_RESP	IK handshake, responder → initiator
0x03	PKT_XX_MSG1	XX handshake message 1
0x04	PKT_XX_MSG2	XX handshake message 2
0x05	PKT_XX_MSG3	XX handshake message 3
0x06	PKT_XX_COOKIE	Stateless retry cookie (return-routability)
0x10	PKT_TRANSPORT	AEAD payload (DHT or control)

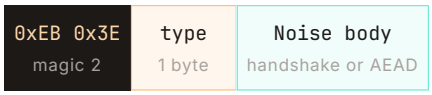

Figure 2. Outer UDP layout. Transport packets add an 8-byte nonce and 16-byte Poly1305 tag around the inner frame (27 bytes of transport overhead including magic and type).

4.2 Return-routability cookie

An unauthenticated XX msg1 can be forged off-path. The responder sends a **stateless retry cookie** (`PKT_XX_COOKIE`) once the unvalidated-handshake budget is spent, so a peer that does not know this packet type still completes first contact whenever the node is not under a flood. The cookie is only useful to a source address that can receive a UDP reply.

Inbound XX handshakes may briefly queue outbound traffic for that peer (honest simultaneous-open). After a 3-second grace the node dials the identity itself; once an IK initiator is pending, further inbound msg1s from that address are refused so a forged handshake cannot be renewed.

4.3 Session lifecycle

LIMIT	VALUE	WHY
`SESSION_TIMEOUT`	900 s	Must outlive the 600 s liveness-ping interval; 300 s forced a fresh handshake on every ping
`MAX_SESSIONS`	4096	LRU eviction when full
`Pending handshakes`	512	Caps unauthenticated work
`MAX_EMBER_DATAGRAM_BYTES`	4096	Hard drop before decryption; sender sees a “successful” send

Sessions are currently keyed by address alone. A shadow-session map can protect an established peer from a key-churning spoofer at the same address, but cannot protect a peer whose *first* contact arrives at an address whose shadow allowance is already full. Keying by `(address, static key)` is listed as future transport work.

4.4 Inner namespace: control vs DHT

Control frames and DHT frames share one decrypted byte stream. The payload is offered to the control decoder first. Control version is `0xC1` so it can never collide with `EMBER_DHT_VERSION` (which counts up from 1). A historical bug: control version 1 made `CONTROL_KIND_EXCHANGE_DATA` (4) alias `MSG_FOUND_NODE` (4), and every iterative lookup stalled after its first hop.

KIND	VALUE	BODY
`CONTROL_VERSION`	0xC1	Leading byte of every control frame
`PING / PONG`	1 / 2	Transport keepalive (not DHT PING)
`EXCHANGE_REQUEST`	3	Empty; ask for UDP EPX payload
`EXCHANGE_DATA`	4	EPX wire payload, packed to the datagram budget

> **EPX IS NOT EMBERDHT**
>
> Ember Peer Exchange (opcode `0xF0` on the eMule extended protocol) is a separate mechanism: source lists exchanged with peers you are already transferring with. It works with the overlay switched off. This specification covers the DHT overlay only.

4.5 Size budgets

A receiver drops anything over 4096 bytes before decryption. Replies whose size we choose are packed to `MAX_UNFRAGMENTED_DATAGRAM = 1400`, matching the spirit of KAD’s `UDP_KAD_MAXFRAGMENT` (1420). Fragmented UDP is dropped by a fair number of consumer NATs; a shorter answer that arrives beats a complete one that does not.

```
MAX_UNFRAGMENTED_PAYLOAD = 1400 - 27 - 120 = 1253 bytes
```

27 = transport overhead (magic + type + nonce + tag). 120 = frame overhead (22-byte header + 32-byte public key + 2-byte length + 64-byte signature). `FOUND_VALUE` *record blobs then have 1253 − 16 − 2 = 1235 bytes after the key and count — about five typical keyword records per reply.*

5. Kademlia parameters and routing

EmberDHT is a 128-bit XOR-metric Kademlia with a replacement cache, verified contacts, and network-size-adaptive diversity caps.

NODE ID	K (BUCKET SIZE)	A (CONCURRENCY)	BUCKETS
128 bit	20	5	128
BLAKE3 of Ed25519 pubkey	KAD uses 10	KAD FindNode uses 1	One per ID bit

CONTACT TIMEOUT	FAILED QUERIES
600 s	3
Then liveness ping	Then evict

5.1 Distance and buckets

```
distance(a, b) = a XOR b   ·   bucket = index of highest set bit of distance (0…127)
```

Bucket 127 holds contacts in the opposite half of the keyspace; bucket 0 holds the nearest neighbours. Node IDs are uniform, so occupancy is geometric: about half of all contacts fall in the last bucket and a quarter in the one before. Diversity caps are designed around that, not around a flat table.

5.2 Contact

A routing-table contact is:

FIELD	SIZE	NOTES
`node_id`	16	Re-derived from ed25519_pub on ingest
`addr`	IPv4 or IPv6 + port	Port 0 is not admitted
`noise_pub`	32	X25519 static key for IK
`ed25519_pub`	32	Signing / identity key
`last_seen`	i64 unix	`> 0` means verified
`failed_queries`	u8	Consecutive unanswered queries

Verified means we have heard a signed frame from the contact directly. Gossip from `FOUND_NODE` / `PEER_LIST` / `ANNOUNCE_PEER` and entries loaded from disk arrive with `last_seen = 0`. They are kept as leads but are not preferred for seeding lookups, answering peers, persisting, or computing network scale.

5.3 Replacement cache

Each bucket holds up to k contacts plus a k-entry replacement cache. When the bucket is full, a new contact is cached and the caller is asked to ping the oldest live entry. If that ping fails, `evict_and_replace` swaps in the newest cache entry. KAD in this codebase has no replacement cache.

5.4 Admission

A contact is refused if any of the following hold:

- Port is 0, or the address is filtered by the user's `ipfilter.dat`.
- Private / LAN / CGNAT addresses while "block private IPs" is on (same preference as KAD; toggling it on evicts already-admitted matches).
- The per-IP or per-subnet diversity cap for the current `NetworkScale` is full (see §12).
- The Ed25519 key is not a valid curve point, or `BLAKE3(key)[..16]` disagrees with the advertised ID.

5.5 Store proximity

A node accepts a STORE only if it is among the k closest contacts it knows of to the key — or if it knows fewer than k contacts, in which case it is among the k closest by definition. Distance is XOR against the local ID compared to the k-th closest contact. Gossip does not count: letting unverified leads into the comparison would let an attacker push the node out of its own neighbourhood.

> **SMALL-NETWORK TRAP**
>
> A previous check used "closer than our furthest contact in any bucket," which on a small network made every node trivially among the k closest to everything — until the table filled, at which point replication silently halved. The k-closest rule is what the code uses now.

6. Wire protocol

6.1 Versioning

CONSTANT	VALUE	MEANING
EMBER_DHT_VERSION	2	Wire version this build speaks
EMBER_DHT_MIN_VERSION	2	Oldest version this build can parse

Version 0 is invalid. A frame outside `[MIN, VERSION]` is refused at the version byte rather than becoming a malformed-payload counter that reads like packet loss. Version was bumped to 2 because batched store, its ack, contact-list trimming, and payload limits all changed shape while the byte still read 1 — two peers announced the same version and then misparsed each other.

A future change that only *adds* to the format can lower `MIN_VERSION` instead of raising both. Incompatible peers currently fail cleanly but silently: neither is told why, and they never fold each other into a routing table. There is no upgrade prompt.

6.2 Frame layout

SIGNED DHT FRAME (AFTER NOISE DECRYPTION)

```
version u8 this build: 2
msg_type u8 see table below
request_id u32 LE correlation; 0 is reserved
sender_id 16 B BLAKE3(sender Ed25519)[..16]
sender_pub 32 B optional; present when include_pub_key
payload_len u16 LE ≤ MAX_DELIVERABLE_PAYLOAD
payload N B message-specific
signature 64 B Ed25519 over every preceding byte
```

Public key is included on frames built for first contact (the engine's builders pass `include_pub_key = true`) so a peer who has never seen us can verify the signature and learn our identity. Inside an established Noise session the decoder may be told the key is omitted; the session already authenticated the static key.

Header without public key is 22 bytes: version + type + request id + sender id. Request id 0 is skipped by the allocator so it can never collide with "unset."

6.3 Message catalog

ID	NAME	DIRECTION	PAYLOAD
0x01	PING	→	empty
0x02	PONG	←	optional observed SocketAddr
0x03	FIND_NODE	→	target node ID (16)
0x04	FOUND_NODE	←	contact list
0x05	STORE_RECORD	→	key + record + record signature
0x06	STORE_ACK	←	key (16)
0x07	FIND_VALUE	→	1…8 keys
0x08	FOUND_VALUE	←	key + records (or FOUND_NODE on miss)
0x09	ANNOUNCE_PEER	→	contact list (gossip dump)
0x0A	PEER_LIST	←	contact list
0x0B	PROXY_STORE	→	same body as STORE_RECORD
0x0C	PROXY_STORE_ACK	←	key (16) — buddy accepted, not a DHT STORE_ACK
0x0D	STORE_BATCH	→	up to 64 records for one destination
0x0E	STORE_BATCH_ACK	←	u64 bitmap of accepted positions

6.4 Contact list encoding

Used by `FOUND_NODE`, `ANNOUNCE_PEER`, and `PEER_LIST`. Contacts are already ordered closest-first; the encoder trims the tail to stay inside the unfragmented payload budget.

```
CONTACT LIST
count u8 ≤ 20, then for each contact:
node_id 16 B
addr 1 + 4 or 16 + 2 BE   0x04 = IPv4, 0x06 = IPv6
noise_pub 32 B
ed25519 32 B
```

An IPv4 contact is 16 + 7 + 32 + 32 = **87 bytes**. At 1253 bytes of payload, only **14 IPv4 contacts** fit, so a `FOUND_NODE` never carries a full k-bucket of 20. `MAX_CONTACTS_PER_RESPONSE` is 20 but is not reachable on IPv4. Dropping the redundant `node_id` (the receiver re-derives it) would take a contact from 87 to 71 bytes and yield 17 contacts per reply — a planned wire change, bundled with the next version bump.

The asker is excluded from a `FOUND_NODE` reply: they were just added to our table, their distance to themselves is zero, and leaving them in spent a slot of a size-capped response on the one contact they already have.

6.5 PING / PONG

`PING` is empty. `PONG` may carry the sender's observed address (the address we received the ping from). An empty PONG payload decodes as "no observation" for backward compatibility. Observed-IP voting is specified in §11.

6.6 FIND_NODE / FOUND_NODE

FIND_NODE PAYLOAD

```
target 16 B node ID we want neighbours of
```

The responder returns the closest contacts it has, excluding the asker, packed as a contact list. Iterative use is specified in §8.

6.7 STORE_RECORD / STORE_ACK / PROXY_STORE

STORE_RECORD AND PROXY_STORE PAYLOAD

```
key 16 B
record_len u16 LE ≤ MAX_STORE_RECORD_BYTES
record N B publisher-signed body
record_signature 64 B Ed25519 over record only
```

`STORE_ACK` and `PROXY_STORE_ACK` carry the 16-byte key. `PROXY_STORE` asks a HighID buddy to fan the same publisher-signed source record out as ordinary `STORE_RECORD`s. The buddy's ack means "I accepted the proxy request," not "the DHT stored it." HighID sources must not ride `PROXY_STORE` — that would bypass the anti-reflection bind (§12.3).

6.8 STORE_BATCH / STORE_BATCH_ACK

Publishing a large library one datagram per (record, target) puts frame count in proportion to records, saturates the link, and trips the receiver's per-peer rate limit. Records destined for the same peer travel together, so frame count scales with the number of peers.

STORE_BATCH PAYLOAD

```
count u8 ≤ 64, then for each record:
key 16 B
rec_len u16 LE
record N B
rec_sig 64 B
trailing bytes are a framing error — the whole batch is refused
```

STORE_BATCH_ACK PAYLOAD

```
accepted u64 LE bit i set ⇒ record i was stored
```

Acceptance is per record: the storer re-evaluates proximity and capacity. A count alone forced the publisher to treat "some landed" as "all landed" and retire files that were never stored. The datagram budget caps a real batch at roughly twenty minimum-size records; 64 is a decode-side bound matching the width of the ack bitmap.

6.9 FIND_VALUE / FOUND_VALUE

FIND_VALUE PAYLOAD

```
count u8 1…8
keys 16 B × count
```

FOUND_VALUE PAYLOAD

```
key 16 B the key this reply is for
record_count u16 LE ≤ 300
for each: len u16 LE + blob (record body ‖ 64-byte publisher signature)
```

The searcher walks the primary (longest) keyword key. Additional keys ride along so a peer that holds records can intersect on `file_hash` locally. On a miss the responder sends `FOUND_NODE` with closer contacts, as in classic Kademlia.

Record blobs in the reply are packed from a rotating window per key (§8.4), into the 1235-byte record budget. A declared count the buffer cannot satisfy is a framing error — the whole frame is refused, so a peer cannot smuggle a truncated payload.

7. Records: keywords and sources

Two record types share a header. The DHT key is not a free field: it is bound to content (`keyword_hash` or `BLAKE3(file_hash)[..16]`) and a STORE under the wrong key is rejected.

```
header = type[1] || keyword_or_source_key[16] || file_hash[16] || ember_file_hash[32]
  || file_size[u64 LE] || publisher_key[32] || timestamp[i64 LE] || name_len[u16 LE]
```

Then: UTF-8 file name (`name_len` bytes), then for source records only the contact block.

TYPE	VALUE	DHT KEY	TTL
`RECORD_TYPE_KEYWORD`	0x01	BLAKE3(lowercase keyword)[..16]	24 h
`RECORD_TYPE_SOURCE`	0x02	BLAKE3(eD2K file_hash)[..16]	6 h

7.1 Keyword hash

```
keyword_hash(s) = BLAKE3(s.to_lowercase().as_bytes())[0..16]
```

Tokens shorter than 2 characters are dropped. Multi-word queries are split on whitespace, lowercased, sorted by length descending (most selective first), and de-duplicated. The first hash is the primary walk key; up to seven more ride on `FIND_VALUE` for peer-side intersection. Stemming is not performed.

7.2 Source contact block

Appended after the file name on source records and covered by the publisher signature:

FIELD	SIZE	NOTES
`ip`	4	IPv4, the address downloaders should dial
`tcp_port`	2 LE	eD2K client-to-client
`udp_port`	2 LE	Ember / KAD UDP
`flags`	1	bit 0 firewalled, bit 1 obfuscation, bit 2 relay-capable
`noise_pub`	32	Stashed for future native (Noise) dialing

A downloader uses `ip + tcp_port` over the existing eD2K path. `SourceContact` carries no buddy address or buddy hash, so a firewalled source is discoverable but not Ember-dialable; those peers work today because they are usually also on eD2K/KAD.

7.3 Ember file hash

`ember_file_hash` is a 32-byte BLAKE3 digest of the file (the hash-tree root specified for the dormant native transfer: BLAKE3 of each 256 KiB chunk, then BLAKE3 of the concatenated chunk hashes).

Search hits, DHT source records, and `known.met` / library entries can carry it. A download verifies against it whenever one is available. Deep links without a digest still complete and are hashed for future sharing.

7.4 Why two TTLs

A source record names an address to download from, so it stops being true the moment that peer goes offline. A keyword record only says a file exists under a word, which stays true whoever is online. Sharing one 24-hour TTL meant a departed peer kept being handed to downloaders for the rest of the day. Publishers re-announce sources every 2 hours, so 6 hours survives two missed republishes while clearing a departed peer four times sooner than a 24-hour source TTL would.

Clock skew tolerance on the signed timestamp is **1 hour** into the future; older than TTL is expired. A replayed older copy cannot replace a newer one: `created_at` is kept and compared.

7.5 Store capacity

CAP	VALUE	BEHAVIOUR WHEN HIT
`MAX_RECORDS_PER_KEY`	300	Refuse the newcomer (do not evict incumbents)
`MAX_RECORDS_PER_PUBLISHER_PER_KEY`	45	Refuse; ~15% of the key, matching KAD's ratio (150 of 1000)
`MAX_KEYS`	50,000	Refuse new keys
`MAX_STORE_BYTES`	48 MiB	Evict least valuable (furthest from our ID, then nearest expiry)
`Sources per IP (adaptive)`	8 / 5 / 3	See NetworkScale; firewalled sources attributed to the forwarder

> **IDENTITY IS FREE**
> A per-publisher rule can only ever *withhold* capacity, never move it. An earlier attempt displaced whichever publisher held the most slots when a key was full. Publisher identity is a free keypair, so an arrival with no slots always outranked an established publisher. A few hundred keypairs stripped a healthy keyword to one record per honest publisher, after which the key admitted no one ever again. Filling an empty key with seven identities is still possible; bounding that needs something scarcer than a keypair.

The 45-record publisher cap is network-wide in effect: every storer applies the same cap to the same publisher key. A user sharing 200 files that share a common word gets 45 of them findable under that word, anywhere — and the same allowance applies to their own local store.

7.6 What a FOUND_VALUE blob is

On the wire, each record in `FOUND_VALUE` is `record_body || signature[64]`. The searcher verifies the publisher signature, checks that `keyword_hash` matches the queried key, and drops junk. Charging the per-node result budget on blobs *offered*, not only those accepted, so a peer that sends nothing but junk still hits its own limit.

8. Iterative lookup and search

8.1 Shortlist walk

Both `FIND_NODE` and `FIND_VALUE` use an iterative shortlist, α = 5 outstanding queries, up to k = 20 closest contacts as the frontier.

1. Seed the shortlist from the routing table (verified contacts preferred). Local store may pre-fill FIND_VALUE results, capped at half the result budget so a well-stocked node cannot finish the search from itself alone.
2. Query up to α pending contacts. A node may be asked twice in one search (one loss is not proof it is gone); a third miss marks it failed.
3. On FOUND_NODE, merge closer contacts. On FOUND_VALUE, verify blobs and merge records. Those two outcomes are independent: treating “no closer node” as “query not accepted” used to stall every value lookup until timeout.
4. FIND_NODE converges after α consecutive responses that bring nothing closer, but only once at least k/2 nodes have answered. FIND_VALUE does *not* use this rule: every extra node is another chance at a record the closer ones did not hold. The walk continues until the shortlist is exhausted, the 300-result budget is full, or 60 seconds elapse.

LIMIT	VALUE
`MAX_ACTIVE_SEARCHES`	64
`SEARCH_TIMEOUT_SECS`	60
`MAX_SEARCH_RESULTS`	300
`MAX_LOCAL_SEED_RESULTS`	150 (half the budget)
`MAX_RESULTS_PER_NODE`	75 (quarter of the budget)
`MAX_QUERY_ATTEMPTS`	2
`Queued-query timeout (cold handshake)`	12 s

The 12-second queued-query timeout covers a Noise_XX 2-RTT setup plus the query round trip. Charging a first-contact hop the ordinary budget failed the contact before it could answer — and on a cold table every contact is a first contact.

8.2 Multi-keyword search

Intersection is **sparse**: missing secondary keys are skipped, plus a filename match at emit time. It is not a strict worldwide AND of every keyword. Tokens of length 1 are ignored. Successive queries rotate which window a storer serves (§8.4).

The Search page exposes an **Ember Only** method. **Global** queries Ember alongside KAD and servers, merging into one de-duplicated list. With KAD and eD2K both offline, Global falls back to Ember alone. Downloads also resolve sources on EmberDHT in addition to KAD and servers.

8.3 Streaming results

Ember used to buffer everything and emit on completion, so on a cold table a user waited most of the 60-second cap while KAD hits were already on screen. As of 1.5.3 the keyword search carries a cursor into an append-only result list. A 1-second sweep emits everything past it on the same cadence as KAD: the first record immediately, then every 20. Locally seeded records therefore reach the UI on the first tick.

Batches run through hash de-duplication against what KAD already streamed, so a hash KAD found arrives as an availability update rather than a duplicate row. Only the batch flagged final clears `ember_pending`, and the completion batch is still queued when empty so that happens exactly once. The timeout backstop that reaps an expired search runs *before* the streaming and emit steps in the same sweep, which is what stops a reaped search from being streamed after its `search-complete`.

8.4 Serving window

A peer answers a keyword query with whatever fits the 1235-byte record budget — typically about five records. Packing used to fill from the front of insertion order, so the oldest five were the only ones that node would ever serve.

Successive `FIND_VALUE`s now rotate the served window per key. The cursor advances by how many records the reply actually carried, and only when the reply withholds some — if everything fits, rotation is a no-op. Blob-hash de-duplication on the searcher is content-based, so rotating windows can only gather more. Local seeding does not inherit this packing.

Truncation is counted: `ember_dht_found_value_truncated` and `ember_dht_found_value_withheld`. Truncation near zero means the ceiling is theoretical on the current network. A high withheld-per-truncated ratio is the case that justifies adding `start_position` to the wire (a version bump).

9. Publish and replication

9.1 Publisher cycle

Shared files are published as keyword records (one per token of length ≥ 2) and one source record, and republished on a cycle to stay alive. Publishing starts once the node has contacts. The Library marks a file with an **Ember** badge once a source record is placed — that is the point at which other Ember users can actually fetch it.

CYCLE	INTERVAL	NOTES
`Source republish`	2 h	Persisted in known.met as FT_EMBER_SOURCE_PUBLISH (0xE4)
`Keyword republish`	12 h	Session-only schedule (not persisted)
`Rendezvous advertise`	5 h	Only while reachable and already publishing something
`Publish timeout`	30 s	Wait for STORE_ACK
`MIN_STORE_NODES`	5	Minimum replicas the publisher aims for
`MAX_ACTIVE_PUBLISHES`	128	In-flight publish operations

Keyword publish rate per maintenance tick scales with library size, clamped between 2 and 96 records per tick, aiming to finish a full pass inside the 12-hour cycle at an estimate of 8 keywords per file. Source publish is similarly paced so a large library is not slammed in one tick after a restart.

The last successful source-publish is written to `known.met`. On start, a stamp still inside the 2-hour interval is hydrated back to an `Instant`; a stamp older than the interval, or one that cannot be represented because the process has not been up that long, is omitted and the file is due immediately — republish-too-eager, the safe direction. Previously the schedule was keyed on `Instant` alone, so every restart marked the whole library as never-published.

9.2 Storer-side replication

Nodes that hold a record re-send the identical signed bytes to the current closest nodes every **2 hours** (`EMBER_RECORD_REPUBLISH_SECS = 7200`), up to 200 records per 60-second maintenance tick, using `STORE_BATCH` where possible.

Replication **cannot extend a record's lifetime**. Expiry is derived from the publisher's signed creation timestamp; every recipient computes the same absolute death time. What it buys is churn coverage: copies reaching nodes that joined since the publisher's last round. Each record already has up to 20 replicas and lives at most 24 hours (6 for sources). Hourly replication was Ember's single largest traffic item — roughly 48,000 frames an hour at 200 records × 20 replicas — about double the entire publish load. Two-hourly halves that.

Firewalled source records are not republished by storers: their declared address is unverified, and repeating it would amplify a lie. A `republish_due` flag, rather than an old `Instant`, marks records whose previous attempt never made it onto the wire — `Instant::now() - 24h` is not representable on a

machine that has been up for less than a day, and the saturating fallback stamped the record as *just* republished.

Each maintenance cycle logs an `Ember publish cycle` heartbeat and an `Ember replication cycle` heartbeat (due, selected, queued, re-armed, leftover backlog).

10. Bootstrap and joining

There is no central bootstrap and no shipped address list. A cold node gets in through the following, in rough order of who arrives first.

1. **The KAD rendezvous key.** Ember nodes advertise themselves under one fixed KAD key as an ordinary source record carrying their Noise pubkey tag. A node with a near-empty table runs a plain source lookup there. Re-advertised every 5 hours (matching KAD's source TTL) and only while the node is reachable and already publishing something — a leecher never generates publish traffic purely to list itself. Lookups are spaced 10 minutes apart and stop once the table reaches one k-bucket of verified contacts. Seed string: `ember-dht-rendezvous-v1`, hashed with MD4 into a KAD ID. The derivation is versioned so the key space can be sharded later without a flag day.

2. **The KAD bridge.** Ember peers noticed in ordinary KAD traffic are DHT-pinged so their signed PONG folds them into the routing table. Capped per maintenance cycle; quiet above 20 verified contacts.

3. **eD2K client-to-client sessions.** Peers that advertise the Ember capability bit over a normal eD2K transfer are cached with their UDP port and bridged via Noise_XX when no static key is known. This is the path for a client running with no KAD at all.

4. **DHT gossip and `nodes_ember.dat`.** FOUND_NODE / PEER_LIST / ANNOUNCE_PEER, plus up to 200 persisted contacts, once the node has been online before. Persisted contacts load as unverified leads.

> **COLD JOIN WITHOUT EMULE**
>
> Every path except `nodes_ember.dat` presupposes either a live KAD connection or an eD2K transfer with an Ember-capable peer. A first-run user with KAD off and no servers has no way in. Ember rides eMule's bootstrap by design. Seed lists are deliberately not planned. For a local two-node test where neither side can reach KAD, the harness command `add_ember_dht_contact` is the only way to introduce two nodes directly.

10.1 Explicitly not planned

- Hardcoded `seeds.txt` or DNS SRV seed lists.
- A rendezvous-hosted bootstrap pool — it leaks an identity-to-IP roster of every participant to whoever runs the server. The pool, its endpoint, and the pinned key it verified were deleted because the server and client never agreed on the envelope format.

The Friends rendezvous server is still used for friend NAT traversal and relay. That is a different protocol and a different trust boundary.

11. NAT, observed address, and firewalled peers

11.1 Observed-IP voting

A `PONG` may echo the address the ping was received from. Votes are counted per reporter `/24`, expire after 15 minutes, and require **3 distinct public nets** before the address is confirmed. Private/loopback reporters and private reported addresses are ignored so LAN Sybils cannot vote. At most 64 distinct reported addresses are tracked; the least-recently-updated is dropped.

Without expiry the three-vote threshold was cumulative over the whole process lifetime, so a stale address stayed qualified indefinitely and a genuine address change could not displace it.

11.2 Firewalled publishing

A LowID / firewalled node sets `SOURCE_FLAG_FIREWALLED` on its source record and asks HighID contacts to `PROXY_STORE`. Storers attribute the record to the forwarder for the per-IP source cap, because the declared address is unverified and an attacker could otherwise invent a new address per record — each getting a fresh quota.

The consume side has nothing equivalent to KAD's `TAG_BUDDYHASH` plus `KADEMLIA_CALLBACK_REQ`. Firewalled Ember sources are discoverable but not Ember-dialable. They work today because they are usually also on eD2K/KAD. A buddy field in the source record and a callback message are planned as part of the next wire version bump.

LowID publishing through buddy `PROXY_STORE` is implemented but not yet exercised end to end on a live network.

12. Abuse resistance

12.1 NetworkScale

Every anti-abuse limit has the same tension: strict values keep one host from occupying the table or the store, but on a small network a legitimate peer looks a lot like an attacker. Limits are derived from **verified** contacts — gossip is cheap and proves nothing. Counting leads would let anyone who can talk at us drive the limits to their strict tier while we had reached almost nobody.

TIER	VERIFIED CONTACTS	PER IP	/24 GLOBAL	/24 PER BUCKET	SOURCES/IP	STORES/MIN
`Bootstrap`	< 10	8	20	5	8	300
`Small`	10…79	4	16	4	5	200
`Established`	≥ 80	2	10	3	3	120

Established per-IP is 2 rather than KAD's 1 because two genuine instances behind one NAT is ordinary, and unlike KAD, Ember can tell them apart cryptographically. Established per-subnet matches KAD (10 global, and the per-bucket cap converges on 3). Store rate is counted in **records**, not frames: every record costs two signature verifications whether it arrives alone or batched. Charging per frame let one dense batch buy many times the admitted work of a well-behaved publisher.

Tightening never evicts contacts admitted under a looser tier, so whatever a cold start allows, an adversary present at that moment keeps for the life of the process — in the bucket that matters most, since geometric occupancy puts half of everyone there. The Bootstrap per-bucket cap is therefore kept ≤ k/4.

12.2 Frame rate limits

Applied after the Noise session is up. A two-bucket sliding window approximates a trailing window so a sender cannot straddle a reset and spend twice the limit. Effective sustained rate is about twice the named constant because the window is enforced over a trailing half.

BUDGET	WINDOW	NAMED CAP	KEYED ON
`All DHT frames`	1 s	40	IP
`FIND_NODE / FIND_VALUE`	10 s	60	IP (shared behind NAT)
`STORE records`	60 s	NetworkScale	Verified node ID when known

Lookup budgets stay on the address because resolving identity for every datagram would put a routing-table scan in front of the gate that exists to make junk cheap to reject. STORE budgets use the verified node ID so peers sharing a NAT do not throttle each other.

12.3 Anti-reflection

HighID / direct source records must claim the observed Noise sender IP. Otherwise a peer could point downloaders at a third-party victim (and Ember would be a traffic reflector). Firewalled sources are exempt: they may be stored by a HighID buddy, or from a NAT mapping that differs from the STUN hint. Authorship is still bound by the publisher Ed25519 signature.

Return-routability is required before a node spends anything substantial on a request (the XX cookie is the handshake-level form of the same idea).

12.4 STORE signature replay

Identical STORE frames (same publisher signature) collapse for 60 seconds so a retransmit storm cannot re-verify the same blob forever. A replay only counts if the store still holds that signature: the cache is never cleared by eviction, and treating a displaced record as a replay made `STORE_BATCH` set the accepted bit for a replica that was gone, after which the publisher retired the file.

Cache size 50,000; swept at most once per TTL. An unconditional size check meant a single 64-record batch could walk a 25,000-entry map 64 times.

12.5 Other gates

- Private IPs and user range filter on routing-table admission.
- No reputation scoring on gossip itself — admission is bounded by diversity caps, and a lead that never answers a liveness ping faults out after 3 failed queries.
- Store refusals are counted by cause: verify, signature, timestamp, anti-reflection IP, per-IP cap, publisher cap, per-key cap, proximity.

13. Maintenance

A 60-second tick drives bucket refresh, liveness pings, gossip, publisher cycles, and storer replication. Each task is internally gated on a longer interval, so the cadence itself is cheap.

TASK	INTERVAL / BUDGET	NOTES
`Tick`	60 s	EMBER_MAINT_INTERVAL
`Bucket refresh`	idle > 3600 s, max 3 FIND_NODE / tick	Random target in the idle bucket
`Liveness ping`	unheard > 600 s	8/tick steady; 32/tick while verified < 20. Scales to cover the table inside the stale window, capped at 32. 8 s ping timeout.
`Lead ping reserve`	1 in 4 of the budget	When verified contacts could spend the whole budget, keep promoting gossip
`ANNOUNCE_PEER`	2/tick; 16/tick while starved	Highest-yield cold frame; self-limits above 20 verified
`Storer republish`	2 h, 200 records/tick	STORE_BATCH to current closest
`KAD bridge`	while verified < 20	Same ping budget as starved liveness
`Rendezvous lookup`	every 10 min while verified < 20	Stops at one k-bucket

A ping is the only way an unverified lead becomes usable, so the starved budget is also the join rate. Eight a minute is a sensible steady-state trickle and a poor join. Verification is cheap — one small frame each.

14. Persistence

14.1 nodes_ember.dat

NODES_EMBER.DAT · MAGIC EMB3 (0X454D4233 LE) · VERSION 1

```
magic(4) || version(1) || count(u16 LE) ||
for each: node_id(16) || addr_type(1) || ip(4 or 16) || port(2 BE) || noise_pub(32) || ed25519_pub(32) ||
last_seen(i64 LE)
```

- Up to 200 contacts. Written atomically (tmp + write + sync + rename + dir-sync).
- An empty table never overwrites an existing file — a momentary empty table would otherwise leave the next launch with no way back in.
- On load, `last_seen` is discarded and set to 0. Restoring it would make every entry look verified the moment it loads, so the staleness purge would delete the whole bootstrap set before a single ping went out. Zero also sorts them first for liveness probing.
- Node ID is re-derived from the persisted Ed25519 key; corrupt keys are dropped.

14.2 store_ember.dat

STORE_EMBER.DAT · MAGIC EMS1 (0X454D5331 LE) · VERSION 1

```
magic(4) || version(1) || count(u32 LE) ||
for each: key(16) || created_at(i64 LE) || publisher_key(32) || signature(64) || ip_present(1) || ip(4,
optional) || data_len(u32 LE) || data
```

- Up to 20,000 records. Written on shutdown only — a periodic write of several megabytes was a disk hitch every few minutes. Abnormal exit falls back to replication refilling the store within the hour.
- Only `data` and `signature` are load-bearing on the way back in. Key, publisher, and expiry are re-derived from the signed body. Believing the unsigned fields would let anything that can write the file file a genuine record under the wrong key or with a life it was never granted.
- Records already past TTL are refused, so the file cleans itself up. Bodies larger than 4096 bytes are corruption.

14.3 Other durable state

LOCATION	WHAT
`known.met tag 0xE4`	Last successful Ember source-publish per file
`ember_dht_highwater.json`	Daily and all-time high-water of verified contacts

15. Comparison with eMule KAD

EmberDHT was compared against this repository's own KAD stack, constant for constant. On design it already matches or exceeds KAD operationally in the rows below. Remaining gaps are in §16.

EMBERDHT

- k = 20 replicas
- α = 5 concurrent FindNode
- Source TTL 6 h vs 2 h republish (3× margin)
- Keyword TTL 24 h
- Replacement cache
- Storer-side replication
- Store survives restart
- Ed25519-signed frames and records
- Noise transport
- Adaptive abuse limits
- Rich diagnostics (truncation, reject causes, search quality, high-water)

KAD (THIS REPO)

- k = 10 replicas
- α = 1 for FindNode
- Source TTL 5 h vs 5 h republish (no margin)
- No replacement cache
- No storer-side replication
- No signed overlay identity
- Cleartext UDP (obfuscation optional)
- Fixed caps (1 contact/IP, 10/subnet, 3 sources/IP)
- 150 of 1000 records per sender per key
- Buddy hash + callback for firewalled consume
- FIND_VALUE pagination (start position)

Roughly half the round trips per lookup come from α = 5 against KAD's 1 for `FindNode`. Replica count is double. The diagnostic surface is already richer. The largest genuine functional gap is firewalled consume (§11.2). The largest product gap is that bytes still move over eD2K.

16. Current limits and planned work

16.1 Known limits (user-facing)

- **Content still moves over eD2K.** Native 256 KiB chunks and a BLAKE3 hash tree exist but nothing imports them.
- **Bootstrap depends on eMule** except for a previously persisted contact file.
- **Version mismatches are silent** — clean refusal, no upgrade prompt.
- **Multi-keyword search is approximate** — sparse intersection plus filename match, not a worldwide AND.
- **Serving ceiling ~5 records per reply** — mitigated by window rotation; pagination waits on a version bump.
- **45 records per publisher per keyword, network-wide.**
- **No reputation scoring on gossip** — Sybil pressure is rate-limited, not scored.
- **Firewalled sources are not Ember-dialable.**
- **Not yet exercised end to end:** LowID PROXY_STORE, cold join from an empty contact file with no KAD, republish across a full record TTL on a large library.

16.2 Planned wire changes (one version bump)

These wait on `EMBER_DHT_VERSION` 3 so they can land together:

1. `start_position` **on FIND_VALUE / FOUND_VALUE** — real pagination. Justified if truncation counters show withheld records on the live network after rotation.
2. **Buddy field + callback message** — firewalled consume, the largest functional gap against KAD.
3. **Drop** `node_id` **from wire contacts** — 87 → 71 bytes, 14 → 17 contacts per FOUND_NODE. The ID is BLAKE3 of the key and is re-derived anyway.

Raising `MAX_RECORDS_PER_PUBLISHER_PER_KEY` toward KAD's 150 is blocked on pagination: if a peer only ever serves ~5 records per reply, extra stored records have no way to reach a searcher, and the only certain effect is more storer-replication traffic.

16.3 Future, non-blocking

- Rendezvous-key health monitoring; shard the rendezvous key space once one KAD bucket's 1000-entry cap is in sight.
- Richer keyword indexing (stemming) if recall lags KAD on real libraries.
- Score gossip under Sybil pressure, beyond per-IP and per-subnet caps.
- Transport sessions keyed by `(address, static key)`.
- Surface BLAKE3 verify pass/fail in the transfer UI; wire the native transfer path.
- Longer fuzz / property tests and overnight soak jobs.

16.4 Dormant native transfer

`ember/transfer.rs` specifies a stream of `REQUEST_CHUNKS`, `CHUNK_DATA`, `FILE_INFO`, and `TRANSFER_COMPLETE` over 256 KiB chunks with a BLAKE3 hash tree whose root is the Ember file hash already carried on DHT records. Nothing in the tree imports this module. Wiring it is the largest remaining piece if the goal is a network that does not need the eMule wire at all.

Appendix A. Constant tables

Values as compiled in Ember 1.5.4. Source files in parentheses.

A.1 Overlay

CONSTANT	VALUE	FILE
EMBER_DHT_VERSION / MIN	2 / 2	dht/mod.rs
K_BUCKET_SIZE	20	dht/mod.rs
ALPHA	5	dht/mod.rs
ID_BITS	128	dht/mod.rs
MAX_CONTACTS_PER_RESPONSE	20 (14 IPv4 fit)	dht/mod.rs
CONTACT_TIMEOUT_SECS	600	dht/mod.rs
MAX_FAILED_QUERIES	3	dht/mod.rs
EMBER_MAGIC	0xEB 0x3E	transport.rs
CONTROL_VERSION	0xC1	transport.rs

A.2 Datagrams and payloads

CONSTANT	VALUE	FILE
MAX_EMBER_DATAGRAM_BYTES	4096	transport.rs
MAX_UNFRAGMENTED_DATAGRAM	1400	dht/messages.rs
FRAME_OVERHEAD	120	dht/messages.rs
TRANSPORT_OVERHEAD	27	dht/messages.rs
MAX_UNFRAGMENTED_PAYLOAD	1253	dht/messages.rs
MAX_FOUND_VALUE_RECORD_BYTES	1235	dht/messages.rs
MAX_FIND_VALUE_KEYS	8	dht/messages.rs
MAX_STORE_BATCH_RECORDS	64	dht/messages.rs
MAX_FOUND_VALUE_RECORDS	300	dht/messages.rs

A.3 Time and maintenance

CONSTANT	VALUE	FILE
EMBER_MAINT_INTERVAL	60 s	network/mod.rs
EMBER_BUCKET_REFRESH_SECS	3600	network/mod.rs
EMBER_RECORD_REPUBLISH_SECS	7200	network/mod.rs
EMBER_SOURCE_REPUBLISH	2 h	network/mod.rs
EMBER_KEYWORD_REPUBLISH	12 h	network/mod.rs
KEYWORD_RECORD_TTL	24 h	dht/store.rs
SOURCE_RECORD_TTL	6 h	dht/store.rs
CLOCK_SKEW_TOLERANCE	1 h	dht/store.rs
EMBER_RENDEZVOUS_REPUBLISH_SECS	5 h	network/mod.rs
EMBER_RENDEZVOUS_LOOKUP_INTERVAL	10 min	network/mod.rs
SESSION_TIMEOUT	900 s	transport.rs
STORE_SIG_REPLAY_TTL	60 s	dht/engine.rs
MIN_OBSERVED_IP_VOTES	3 / 15 min	dht/observed.rs

A.4 Persistence and identity files

CONSTANT	VALUE	FILE
EMBER_PERSIST_MAX_CONTACTS	200	network/mod.rs
EMBER_PERSIST_MAX_RECORDS	20,000	network/mod.rs
NODES_EMBER_MAGIC	EMB3	dht/bootstrap.rs
STORE_EMBER_MAGIC	EMS1	dht/bootstrap.rs
FT_EMBER_SOURCE_PUBLISH	0xE4	known.met
EMBER_RENDEZVOUS_SEED	ember-dht-rendezvous-v1	kad/publish.rs

Appendix B. Code map

AREA	LOCATION
DHT engine, routing, store, search, publish, wire	`src-tauri/src/network/ember/dht/`
Protocol constants (k, α, version, IDs)	`dht/mod.rs`
Frame encode/decode, message types, size budgets	`dht/messages.rs`
IO-free inbound handling, proximity, PROXY_STORE	`dht/engine.rs`
128-bucket table, replacement cache, admission	`dht/routing.rs`
Signed records, capacity, TTL, serve cursor	`dht/store.rs`
Iterative lookup, keyword hashing	`dht/search.rs`
Keyword/source record builders	`dht/publish.rs`
Adaptive abuse limits	`dht/scale.rs`
Per-IP / per-peer rate limits	`dht/protection.rs`
Observed-IP voting	`dht/observed.rs`
nodes_ember.dat / store_ember.dat	`dht/bootstrap.rs`
Noise transport, magic, sessions	`ember/transport.rs`
Node ID, sign/verify, hash binding	`ember/crypto.rs`
Dormant native transfer	`ember/transfer.rs`
Publish/search drivers, maintenance, KAD bridge	`network/mod.rs`
Rendezvous KAD key	`kad/publish.rs` · `ember_rendezvous_key()`
User-facing status	Ember Network page (`/ember`), status bar

AREA	LOCATION
Settings	Settings → Network · `ember_native_enabled` (always on)
Dev harness	`add_ember_dht_contact`, `ember_dht_ping_peer`, `ember_dht_find_node`, `ember_dht_iterative_find_node`, `ember_dht_publish_keyword`, `ember_dht_find_value`, `ember_dht_run_maintenance`
Standing work log	`docs/ember-dht.md`

EmberDHT Protocol Specification · Wire version 2 · Ember 1.5.4 · August 2026. Derived from the canonical implementation in the ember-p2p repository. Licensed under the GNU General Public License, version 3.

This document describes behaviour as implemented. It is not a promise that the wire will remain unchanged: version 3 is reserved for pagination, firewalled consume, and contact-list compaction. Implementers targeting interoperability should treat `EMBER_DHT_MIN_VERSION..=EMBER_DHT_VERSION` as the only supported range.